
A BENCHMARK FOR AI-ACCELERATED OCEAN SPIN-UP

A PREPRINT

✉ Matt Archer^{1,†}, Isaac Akanho¹, Simon Sadler¹, Surbhi Goel¹,
Etienne Meunier², Guillaume Gachon², Maud Tissot², David Kamm^{1,*}, Julie Deshayes²

¹Institute of Computing for Climate Science (ICCS), University of Cambridge, UK

²LOCEAN, Institut Pierre-Simon Laplace (IPSL), Sorbonne Université, France

*Work carried out while a PhD student at LOCEAN/IPSL under the supervision of J. Deshayes.

†Correspondence: mattjarcher3@gmail.com, iccs@cam.ac.uk

June 27, 2026

ABSTRACT

Spinning up an ocean general circulation model to a statistical equilibrium is among the most computationally expensive steps in climate modelling, often requiring thousands of simulated years to reach a drift-free baseline. A growing body of work uses machine learning to shortcut this procedure by predicting equilibrated ocean states, yet comparisons across methods remain difficult: each study uses its own configuration, forcing, diagnostics, and restart-file handling. We present **nemo-spinup-bench**, an open benchmark for AI-accelerated spin-up of the NEMO ocean model. The benchmark decomposes the problem into three reusable components: forecasting, restart-file construction, and physics-based evaluation, each exposed as a standalone Python package with a pluggable strategy interface, reproducible pipelines, and continuous integration. We ship reference baselines (PCA with Gaussian-process forecasting) on the idealised DINO configuration, along with a published dataset of NEMO outputs and restarts. By separating the ML forecaster from the surrounding numerical machinery, the benchmark lets new methods (including deep generative models and neural emulators) be evaluated against a common yardstick, with planned extensions to OM4 and Oceananigans.

Keywords Ocean spin-up · NEMO · Benchmark · Machine learning · Climate modelling · DINO

1 Introduction

Recent progress in AI-driven oceanography spans neural general circulation models (?), global ocean emulators (?), and deep generative approaches for producing physically consistent ocean states (??). Progress in this area is increasingly supported by open benchmarks such as OceanBench for short-range forecasting (?), and by shared idealised model configurations such as the DINO diabatic pole-to-pole setup (?) used throughout this paper.

‘Spin-up’ is the process of running an ocean or climate model forward from an arbitrary initial state (typically no circulation and a present-day climatological temperature and salinity) until it reaches a statistical equilibrium suitable for analysis. It is one of the most computationally expensive steps in climate modelling: reaching equilibrium for a global ocean model takes on the order of 0.5M CPU-hours, and must be repeated for every new experiment. At IPSL alone, roughly five such experiments per year consume on the order of 2.5M CPU-hours purely on spin-up, before any scientific integration begins. The same pattern holds at every modelling centre, and for the land component as well as the ocean. Even modest acceleration of spin-up would therefore free substantial compute for science.

Accelerating spin-up is simultaneously a machine-learning problem, generating high-dimensional ocean states that are physically consistent and closer to equilibrium than the standard initialisation, and a systems problem, delivering those states back into a running ocean model in a format it will accept. The ML side is a natural fit for modern generative modelling and climate emulators (????), yet progress has been slowed by the absence of a shared task. Image and video generation advanced as fast as they did in part because benchmarks like ImageNet, DAVIS and Kinetics gave the

community a common yardstick; no equivalent exists for ocean spin-up, in large part because evaluating a candidate state requires tight coupling to an ocean model, which is unfamiliar territory for most ML researchers.

This paper introduces **nemo-spinup-bench**, an open benchmark for AI-accelerated spin-up of the NEMO ocean model. Our contributions are:

- A decomposition of the spin-up task into three reusable, independently installable components (forecasting, restart-file construction, and physics-based evaluation), so that ML researchers can plug in a new forecaster without touching NEMO internals, and ocean modellers can reuse the restart and evaluation tooling with any forecaster.
- A published Zenodo dataset of NEMO/DINO outputs and matching annual restart files, together with reference baselines, reproducible pipelines, and continuous integration across all components.
- A reference baseline (PCA with Gaussian-process forecasting) that recovers the method of ?, and a pluggable architecture that admits deep-learning forecasters as drop-in replacements.

The remainder of the paper is organised as follows. Section ?? situates the work against neural GCMs, ocean emulators, and generative approaches to ocean-state initialisation. Section ?? defines the spin-up task and convergence criteria. Section ?? describes the benchmark components, data, and metrics. Section ?? details the shipped baselines, and Section ?? reports headline results.

2 Related work

3 The spin-up problem

4 Benchmark design

nemo-spinup-bench defines a reproducible, end-to-end task for evaluating machine-learning-based acceleration of ocean spin-up. The current benchmark is built around the DINO idealised diabatic configuration of NEMO (?), which offers the minimum complexity required to exhibit the large-scale circulation patterns relevant to spin-up while being small enough to permit rapid iteration. Extensions to OM4 and Oceananigans are planned as future work, and the component interfaces have been designed so that swapping the underlying ocean model amounts to providing a new variable-to-file mapping and a compatible restart writer, rather than altering the pipeline itself. A dataset containing the DINO simulation outputs, restart files, and reference evaluation outputs is published on Zenodo (?) and is pulled in as a submodule by the benchmark repository, so that any reported result is tied to a specific, citable snapshot of the data.

The benchmark deliberately fixes the task rather than the method: given a simulation window of N training time units, a method must produce a projected ocean state M time units in the future (the cadence is set by the input data: yearly, monthly, or other), from which a valid NEMO restart file is assembled and evaluated. Every component between the input window and the final restart is pluggable. The default pipeline ships a PCA + Gaussian process baseline, recovering the behaviour of the original spin-up tool (<https://github.com/maudtst/Spinup-NEMO>), but any alternative dimensionality-reduction or forecasting strategy can be substituted by registering a new class and selecting it through a YAML configuration (Section ??).

4.1 Pipeline

The benchmark is composed of three interoperable tools, each released as an installable Python package with its own test suite and continuous integration:

- **nemo-spinup-forecast** performs dimensionality reduction on the training window, fits a forecaster in the reduced space, and reconstructs predictions for sea-surface height, temperature, and salinity. Each run writes to a timestamped, content-addressed output directory, so experiments accumulate without clobbering each other.
- **nemo-spinup-restart** consumes these predictions together with the mesh mask and a source restart and derives the remaining prognostic fields required by NEMO (density, velocities, surface-mean diagnostics), producing a restart file that NEMO can pick up without modification. An upsampling path based on xESMF bilinear interpolation (?) allows coarse-resolution predictions to be applied to finer target grids.

- **nemo-spinup-evaluation** scores the resulting restart against a physically-motivated reference run, computing a suite of domain metrics and optionally recomputing diagnostic quantities such as potential density from the underlying temperature and salinity fields (?).

The default example drives the full pipeline on a 50-year DINO window: baseline grids and restarts are first evaluated, a thirty-year forecast is produced from the 20 → 50 year segment, a new restart is assembled, and the final state is re-evaluated against the reference. A second example applies the same pipeline to previously published diffusion-based predictions (?), and reproduces the reported metrics to within a floating-point tolerance, providing an external check on the implementation.

4.2 Metrics

Evaluation combines standard error metrics against a reference run with a suite of physics-based diagnostics that target specific features of the large-scale circulation and water-mass structure. The diagnostics shipped with `nemo-spinup-evaluation` are:

- **Density monotonicity.** Proportion of grid points at which the in-situ density profile fails to increase with depth, used as a proxy for static stability of the water column.
- **Thermocline temperature** ($T_{500\text{ m}}, 30^{\circ}\text{S}-30^{\circ}\text{N}$). Volume-weighted mean temperature at 500 m depth in the tropics, a sensitive indicator of upper-ocean heat content and thermocline drift.
- **Bottom-water and deep-water temperature boxes.** Volume-weighted mean temperature in a U-shaped “Bottom Water” box (below 3000 m or poleward of 30°) and a complementary “Deep Water” box, targeting the long-timescale equilibration of the abyss.
- **ACC transport (Drake Passage).** Barotropic volume transport through the Drake Passage section, a canonical measure of the strength of the Antarctic Circumpolar Current.
- **North Atlantic subtropical gyre strength.** Maximum of the barotropic streamfunction in the North Atlantic subtropical gyre region, summarising wind-driven gyre intensity.

A full description of the metrics, including implementation details and the handling of coordinates, lazy loading, and temporal alignment, is given in the `nemo-spinup-evaluation` documentation at <https://nemo-spinup-evaluation.readthedocs.io/en/latest/>.

5 Baselines

The forecasting framework ships two registries of interchangeable components (one for dimensionality reduction and one for forecasting) which together define the baselines shipped with the benchmark. Both registries are exposed as `name → class` dictionaries and selected at runtime through `techniques_config.yaml`.

Dimensionality reduction. Two implementations are provided out of the box:

- **PCA** (`DimensionalityReductionPCA`): linear principal-component analysis applied to the masked, standardised ocean field. The number of retained components is either a fixed integer or a target fraction of explained variance, controlled by the `-comp` CLI argument.
- **Kernel PCA** (`DimensionalityReductionKernelPCA`): a non-linear variant using `scikit-learn`’s kernel PCA, intended as a drop-in reference for non-linear reductions.

Forecasting. Two Gaussian-process variants are registered, both built on a shared kernel following the original formulation by Tissot and co-authors. The two wrappers differ only in how this regressor is invoked:

- **GaussianProcessForecaster:** a direct forecaster that fits the GP once on the full training window and predicts all future steps in a single call. Returns both point predictions and posterior standard deviations.
- **GaussianProcessRecursiveForecaster:** a recursive forecaster built on `skforecast`’s `ForecasterRecursive`, which feeds lagged targets and rolling window features back into the regressor to produce multi-step forecasts one step at a time. Uncertainty estimates are not available in this mode.

Data. The baselines are driven by a published DINO dataset hosted on Zenodo (?). Two archives are used: `200.zip` contains 200 years of annual-mean DINO output on the native grid (sea surface height, temperature, salinity, and companion fields on the T, U, V grids, together with mesh masks), and `restart.zip` contains the matching 200 annual restart files needed to seed a continuation run of NEMO. Forecasting is performed on a contiguous training window from `200.zip`, and the predicted final state is combined with the corresponding restart from `restart.zip` to assemble a physically valid restart that NEMO can read directly. A smaller `50.zip` (the first 50 years) is provided for quick smoke tests and continuous-integration runs.

Default pipeline. The packaged `techniques_config.yaml` selects **PCA** as the dimensionality-reduction technique and **GaussianProcessForecaster** as the forecaster, recovering the method of Tissot et al. These are the baselines used to produce the headline results in Section ???. Alternative combinations (PCA with the recursive GP, Kernel PCA with either forecaster) can be selected by editing a single YAML entry; adding a new baseline amounts to subclassing the appropriate base class and registering it, without changes to the CLI or the pipeline.

6 Results

7 Discussion

8 Conclusion

Data availability

The DINO dataset used throughout this paper is archived on Zenodo at <https://doi.org/10.5281/zenodo.19557419> (?). It contains the NEMO/DINO simulation outputs and restart files needed to reproduce the benchmark, together with the reference outputs used for evaluation.

A Software architecture

The benchmark is implemented as four loosely-coupled repositories (`nemo-spinup-bench`, `nemo-spinup-forecast`, `nemo-spinup-restart`, and `nemo-spinup-evaluation`), each released as an installable Python package with a `src/` layout, a `pyproject.toml`, and a console entry point. The separation reflects the fact that the same forecasting method might be paired with different restart-assembly strategies when targeting different ocean models, and that the evaluation tooling is independently useful for comparing any pair of ocean simulations. A Zenodo-backed dataset of DINO outputs and reference restarts (?) is consumed by all four repositories as a submodule, so that the data, code, and metrics reported in this paper are reproducible from a single commit graph.

Pluggable strategies. Dimensionality reduction and forecasting are expressed as abstract base classes (`DimensionalityReduction` and `BaseForecaster`) with small interfaces. Concrete implementations (PCA, kernel PCA, direct and recursive Gaussian processes) are registered in a dictionary and selected by name through `techniques_config.yaml`. Adding a new strategy, including deep-learning forecasters built on PyTorch or JAX, requires only subclassing the relevant base and registering the class; no changes to the orchestration code or the CLI are needed. Variable-to-file binding is likewise externalised to `ocean_terms.yaml`, so different model configurations can be targeted without code changes.

Reproducible runs. The forecasting CLI writes each run to a timestamped, content-hashed directory under the user-supplied output path, with a `latest` symlink pointing at the most recent run. This keeps experimental artefacts isolated and makes bisection over code or configuration changes straightforward. Pipeline orchestration is driven by a list of `TermDef` specs: small immutable dataclasses that bundle an internal key, a NetCDF variable name, and a source filename, so new variables can be added to the pipeline by appending a single entry.

Continuous integration. Each repository enforces formatting and linting with `ruff`, runs pre-commit hooks on every push, and executes its full `pytest` suite on every pull request. Test fixtures that require real NetCDF inputs are fetched from Zenodo. The evaluation package additionally publishes versioned API documentation to Read the Docs on successful CI.

Test coverage. The combined test suites exercise PCA round-trip identity to $\approx 10^{-5}$, the NEMO restart-variable derivation, coordinate handling on real DINO NetCDFs, and the end-to-end pipeline from raw outputs to final metrics. Fixtures parametrise tests over every registered strategy, so introducing a new dimensionality-reduction or forecasting technique automatically expands the coverage matrix.